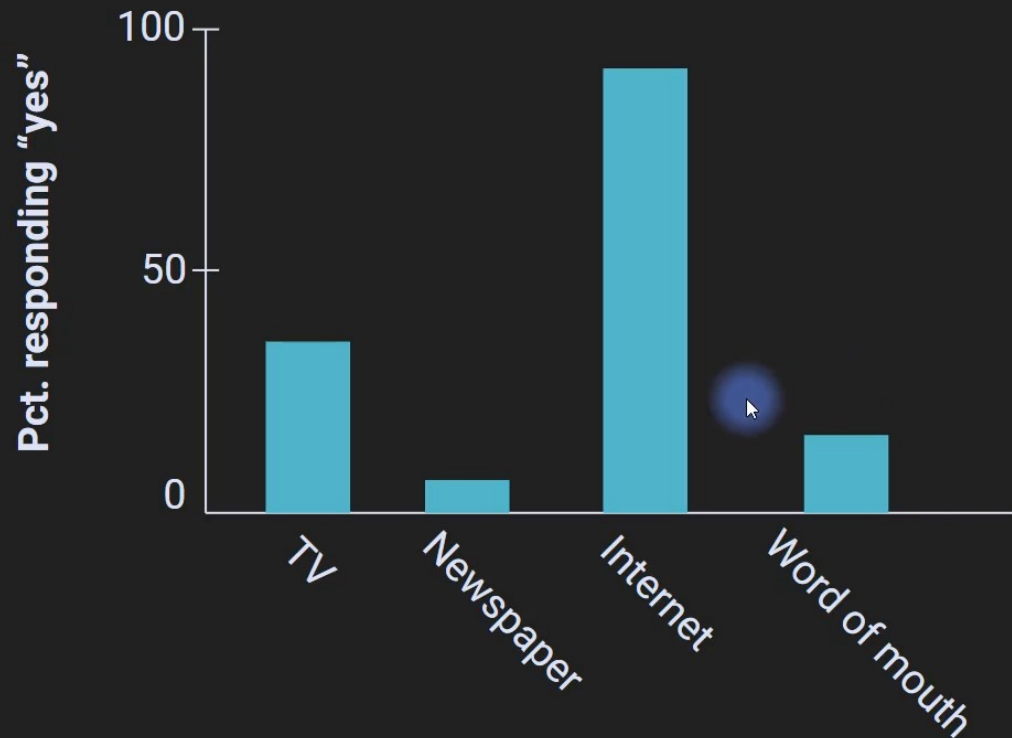


The bar plot

How do people get the news?



What kind of data are these?

Nominal!

What kinds of data can be shown in a bar plot?

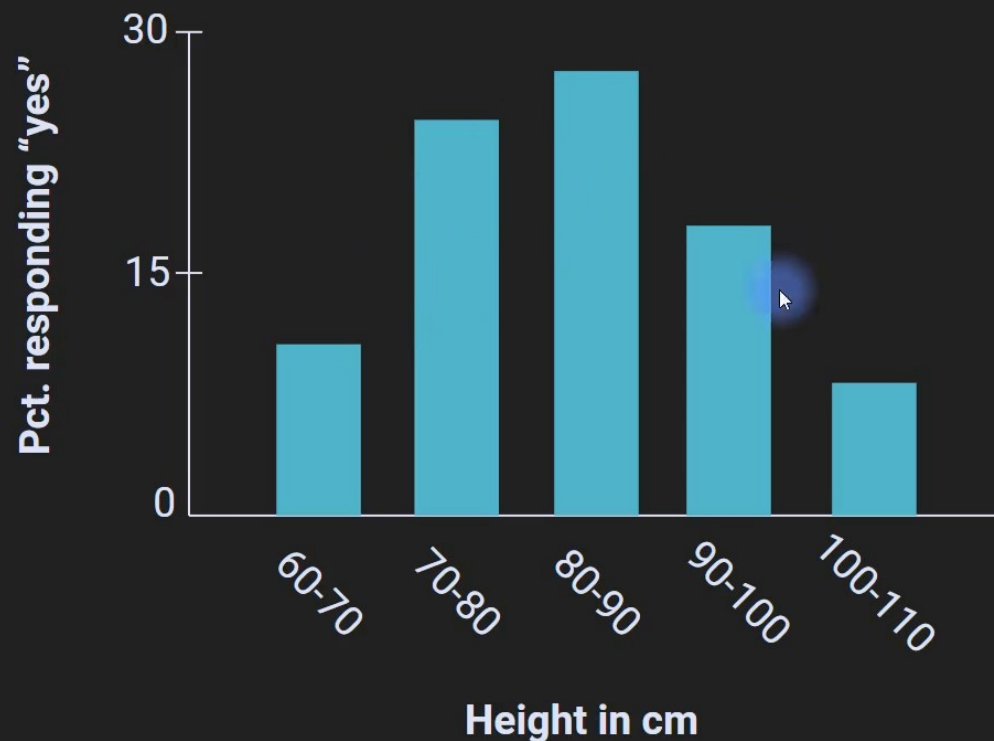
Categorical (nominal and ordinal)
Numerical (only discrete)

Numerical data in bar plots



https://en.wikipedia.org/wiki/King_penguin

How tall are king penguins?



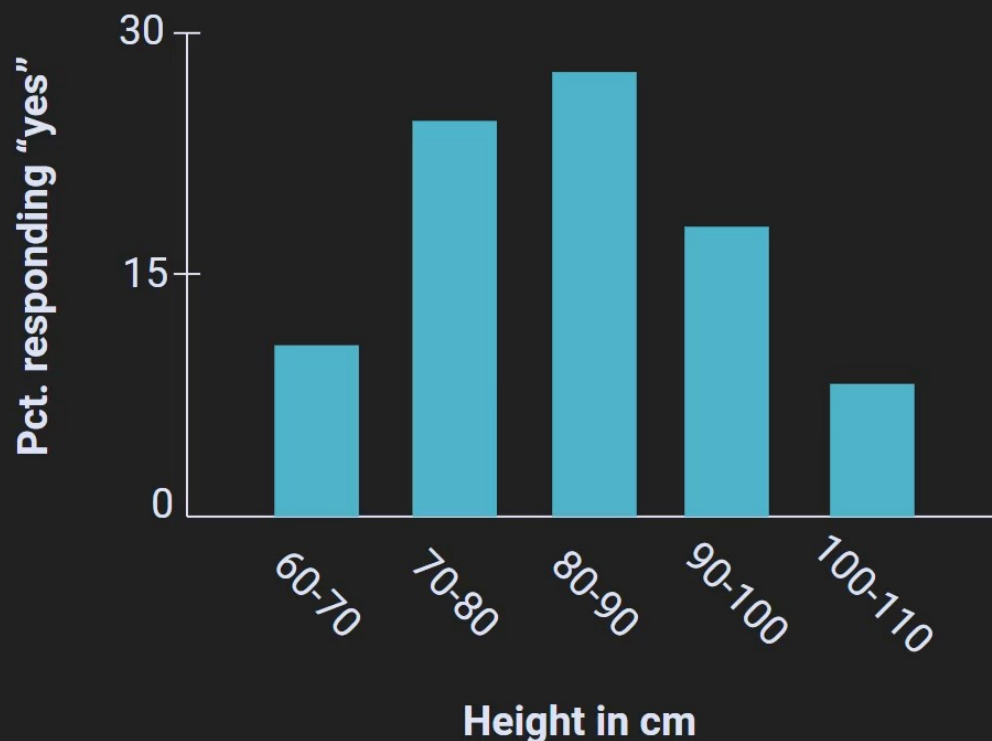
Numerical data in bar plots

Conclusions:

Numerical data must be converted to discrete type for bar plot visualization.

Bin number and boundaries are important (more on this point in later videos).

How tall are king penguins?



Numerical data in bar plots

How tall are king penguins?

Conclusions:

Numerical data must be converted to discrete bins for bar plot visualization.

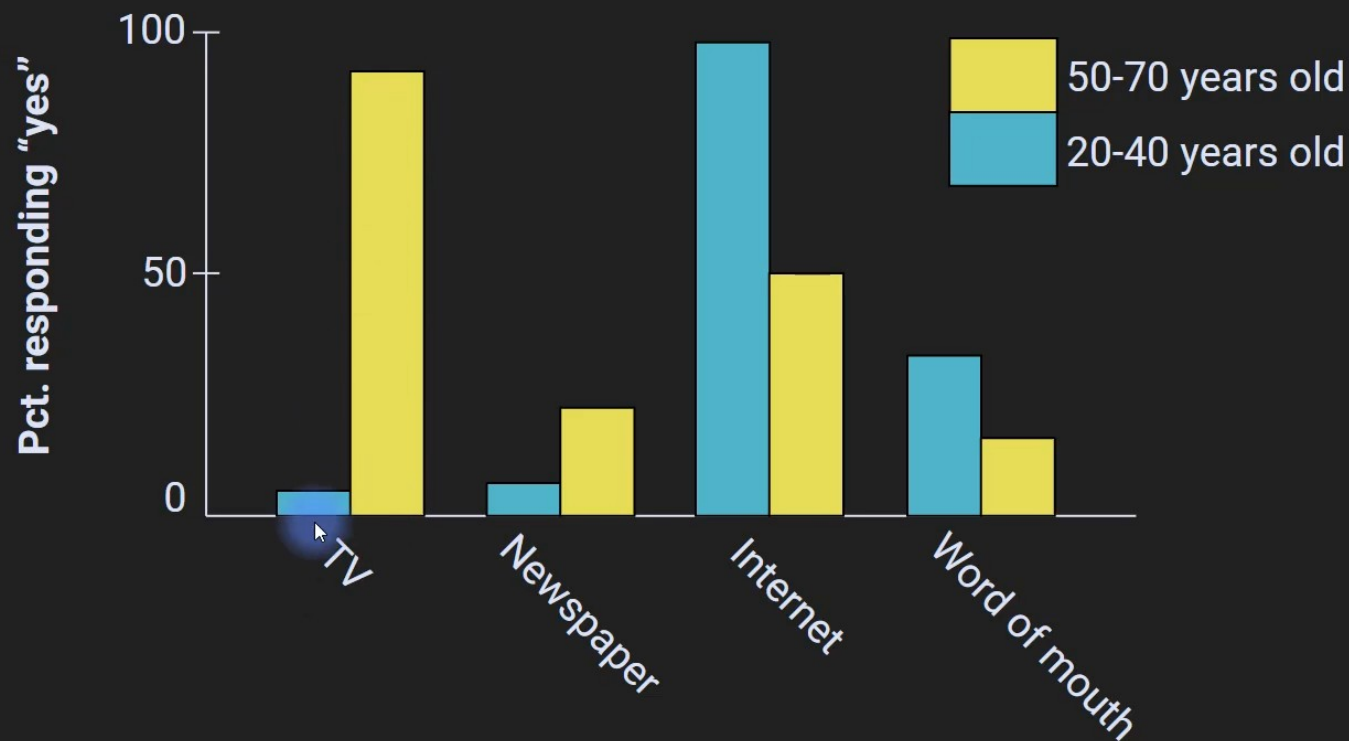
Bin number and bin width are important (more in later videos).

Note: technically this is a histogram, not a bar plot, but the terms are often used interchangeably.



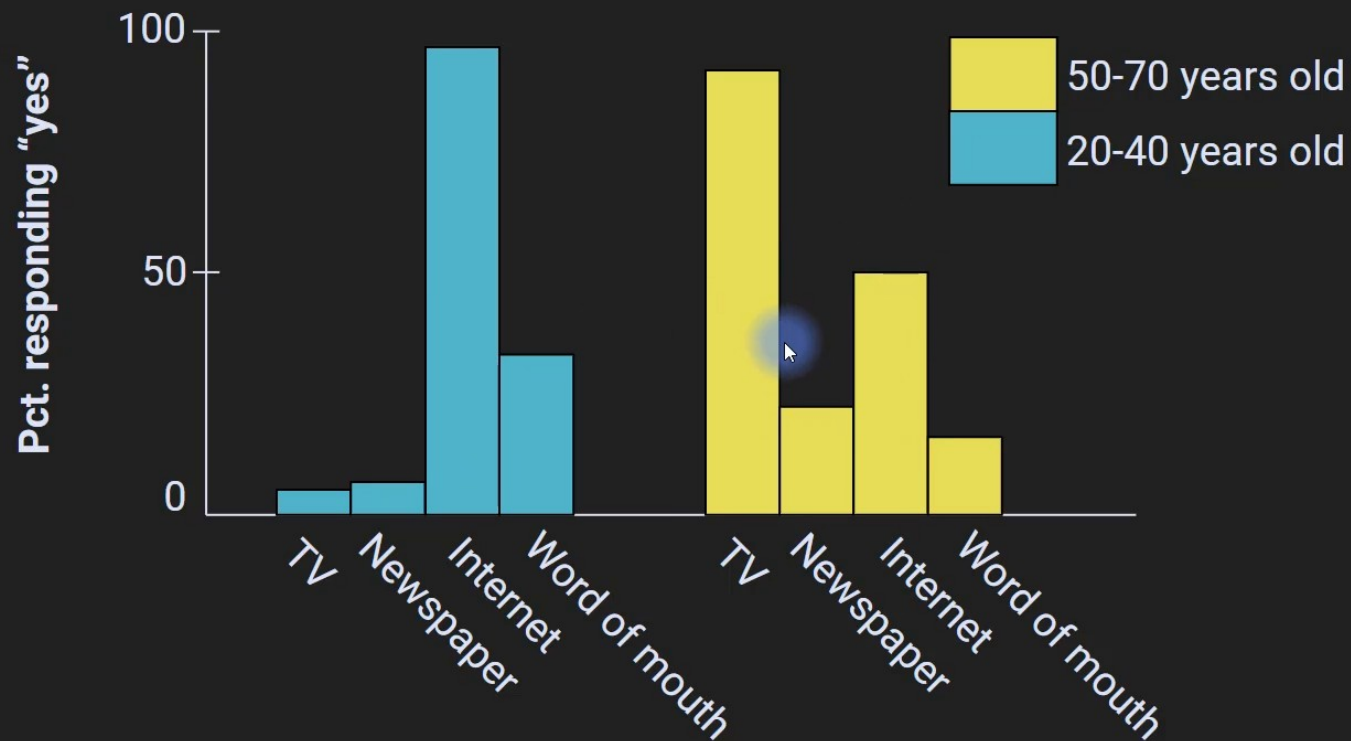
The bar plot

How do **old** vs. **young** people get the news?



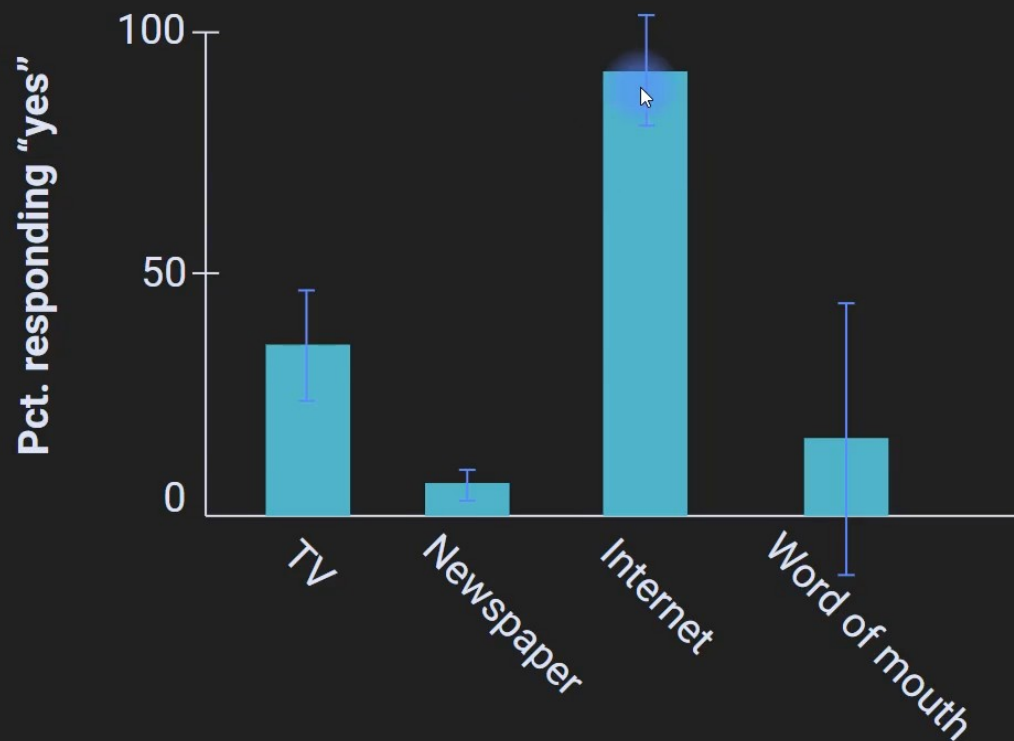
The bar plot

How do **old** vs. **young** people get the news?



Error bars

How do people get the news?



What do error bars show?

Standard deviation, standard error, or confidence intervals

Can error bars be misinterpreted or used for evil purposes?

Yes! (To the first question.)

It's important to use error bars correctly and explain them clearly. More on this later in the course.

Bar plots

In this video, you have learned

- How to interpret bar plots.
- The types of data that can be used in bar plots.
- Best practice for organizing and labeling bar plots.
- Introduction to error bars.

jupyter stats_visualizing_barplots (at

File Edit View Insert Cell Kernel W



VIDEO: Bar plots

TEACHER: Mike X Cohen, sincxpress

```
In [1]: # import libraries
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
```

```
In [ ]: ## create data for the bar plot

# data sizes
m = 30 # rows
n = 6 # columns

# generate data
data = np.zeros((m,n))

for i in range(n):
    data[:,i] = 30*np.random.randn(m) * (2*i/(n-1)-1)**2 + (i+1)**2
```

```
In [ ]: # show the bars!

fig,ax = plt.subplots(1,3,figsize=(8,2))

# 'naked' bars
```

```
[[ 1.53438143e+01  6.88064193e+00  9.05797105e+00  1.59112209e+01  1.32436532e+01  4.28803818e+01]
 [-1.25352732e+01  1.69403419e+01  8.47954025e+00  1.56448957e+01  4.41531905e+00 -3.22859194e+01]
 [-8.69493362e+00  7.18850426e+00  7.79733146e+00  1.80432543e+01  2.57664773e+01  2.47045037e+01]
 [ 5.04856313e+01  3.91246959e+00  7.67953061e+00  1.51326437e+01  3.36222839e+01  9.35833982e+00]
 [-3.93860644e+01 -7.22947115e-01  1.01757723e+01  1.54975974e+01 -6.79173707e-04  1.21955302e+01]
 [-1.56296790e+01  9.39359853e+00  9.37215950e+00  1.64031477e+01  2.60765183e+01  3.98563117e+01]
 [ 4.23774527e+01 -3.08920297e-01  1.00194269e+01  1.76623829e+01  3.44161073e+00 -1.70231783e+00]
 [-1.70146901e+01  5.79183028e+00  9.83510517e+00  1.54955155e+01  3.59187086e+01  3.19237038e+01]
 [ 1.36027310e+01  1.31408106e+01  1.06575582e+01  1.61274779e+01  1.43599864e+01 -2.71434227e+01]
 [ 1.52559540e+01 -8.54900763e+00  7.82172593e+00  1.70466377e+01  3.69867264e+01  6.77271183e+01]
 [-3.58286839e+01  7.58659485e+00  9.80568031e+00  1.71918914e+01  3.85634965e+01  1.06539462e+02]
 [-2.65289732e+01  5.00932278e+00  9.65481852e+00  1.54964747e+01  1.61008087e+01  1.68892445e+01]
 [-4.58262492e+00  1.34190016e+01  1.01025706e+01  1.58538638e+01  3.62820544e+01  2.81508025e+01]
 [-3.06465576e+01 -6.79347054e+00  7.14636494e+00  1.58791015e+01  3.50713423e+01  8.70143665e+01]
 [ 2.27215134e+01  3.29604220e+00  7.17600952e+00  1.67344062e+01  1.65264369e+01  3.68550757e+01]
 [-2.64728938e+01  4.53456971e+00  9.04153197e+00  1.57587575e+01  2.31545738e+01  4.61467199e+01]
 [ 9.96804136e+00  5.12830225e+00  1.03231116e+01  1.61948769e+01  1.71348574e+01  8.73945392e+01]
 [ 1.47155641e+00  3.33891630e+00  8.60261527e+00  1.59680799e+01 -3.35135619e+00 -2.57088396e+01]
 [ 1.40882197e+01  1.36202875e+01  8.66434656e+00  1.60554076e+01  2.63445249e+01  7.46674566e+01]
 [ 1.28391288e-01  4.77767651e+00  8.59448502e+00  1.57297656e+01  1.29543469e+01 -2.13761113e+01]
 [ 2.95149491e+00 -1.99041294e+01  7.85087532e+00  1.87161184e+01  2.72344827e+01 -1.02881852e+01]
 [ 3.88746996e+01 -7.89477091e+00  1.02461534e+01  1.55368348e+01  1.62414967e+00  6.34781510e+01]
 [ 1.95067029e+01  7.98564156e+00  6.91640128e+00  1.63918406e+01  2.03224243e+01  1.86925700e-01]
 [-4.63614619e-01  1.65603912e+01  7.62576525e+00  1.53904693e+01  1.17367636e+01  3.46222411e+01]
 [ 1.14898156e+00  8.06241116e+00  9.60848751e+00  1.73803282e+01  2.34140304e+01  8.46627656e+01]
 [-2.40171871e+01  1.58718578e+01  9.87410308e+00  1.52808830e+01  3.17137728e+01  1.51296622e+01]
 [ 3.50615377e+01 -9.66155098e+00  7.01674477e+00  1.75003242e+01  4.03314914e+01 -1.25599816e+01]
 [ 3.38629591e+01  6.94064695e-01  8.60349186e+00  1.55670344e+01  2.94966108e+01  6.15725622e+01]
 [ 8.60197585e+00 -1.23232668e+01  9.96866303e+00  1.77559382e+01  1.5135449e+01  1.51267611e+01]
 [-1.71701342e+01 -1.12348670e+01  1.14551408e+01  1.48574760e+01  1.71008113e+01  1.42756832e+01]]
```

```
In [ ]: # show the bars!
```

```
fig,ax = plt.subplots(1,3,figsize=(8,2))
```

```
# 'naked' bars
```

```
ax[0].bar(range(n),np.mean(data,axis=0))
```

```
ax[0].set_title('Bar plot')
```

```
# just the error bars
```

```
ax[1].errorbar(range(n),np.mean(data,axis=0),np.std(data,axis=0,ddof=1),marker='s',linestyle='')
```

```
ax[1].set_title('Errorbar plot')
```

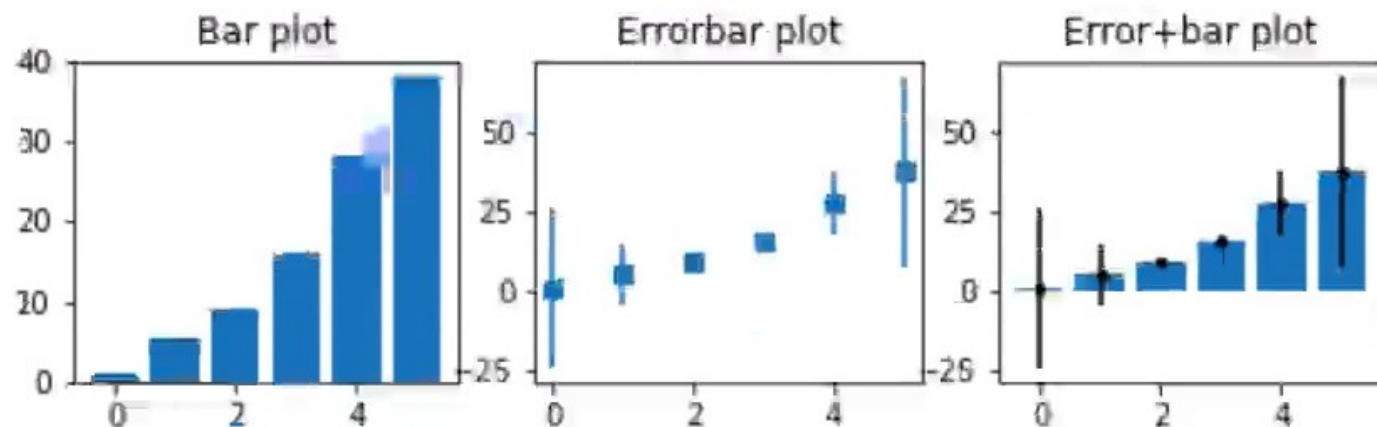
```
# both
```

```
ax[2].bar(range(n),np.mean(data,axis=0))
```

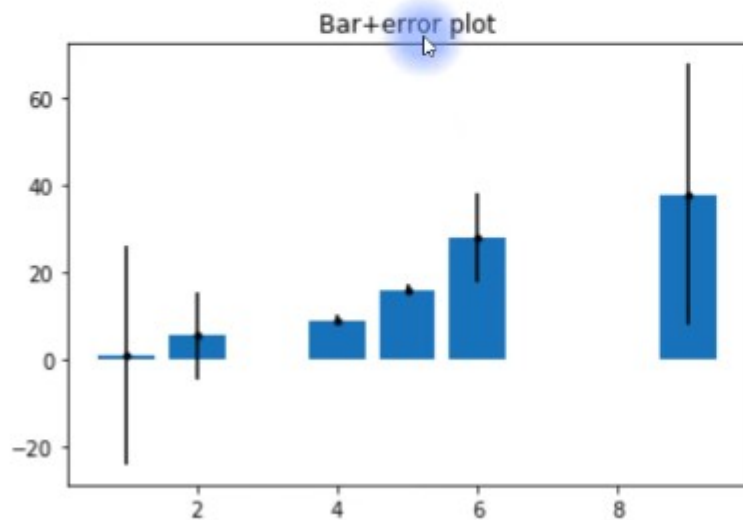
```
ax[2].errorbar(range(n),np.mean(data,axis=0),np.std(data,axis=0,ddof=1),marker='.',linestyle='',color=
```

```
ax[2].set_title('Error+bar plot')
```

```
plt.show()
```



```
xcrossings = [ 1, 2, 4, 5, 6, 9 ]  
  
plt.bar(xcrossings,np.mean(data,axis=0))  
plt.errorbar(xcrossings,np.mean(data,axis=0),np.std(data,axis=0,ddof=1),marker='.',linestyle='',color=  
plt.title('Bar+error plot')  
  
plt.show()
```



```
In [ ]: ## note about bars from matrices

# data are groups (rows) X property (columns)
m = [ [2,5,4,3], [1,1,8,6] ]

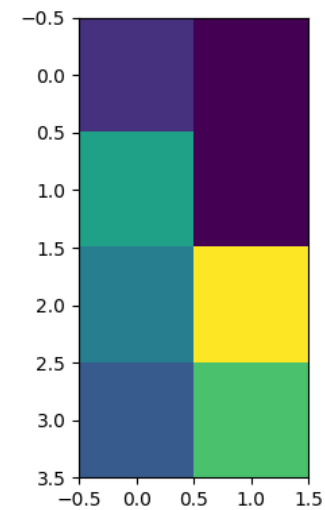
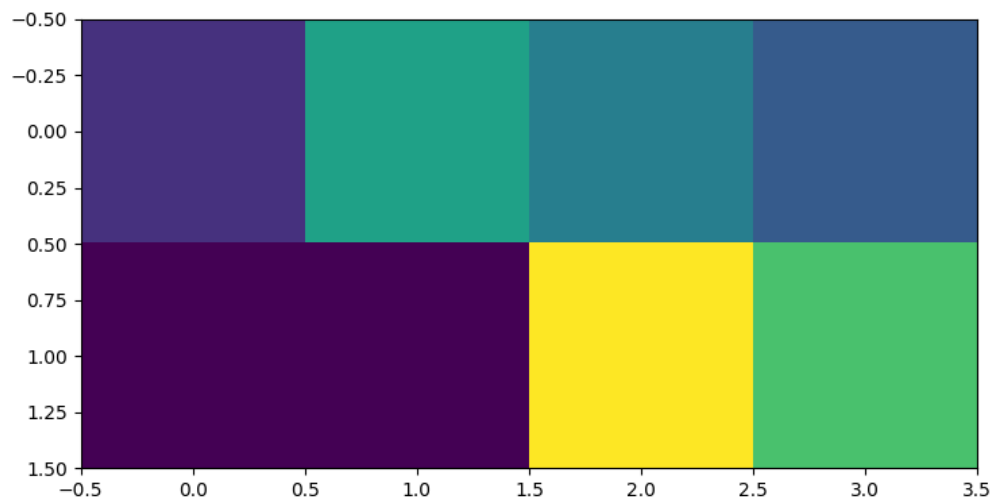
fig,ax = plt.subplots(nrows=2,ncols=2,figsize=(8,8))

# conceptualizing the data as <row> groups of <columns>
ax[0,0].imshow(m)

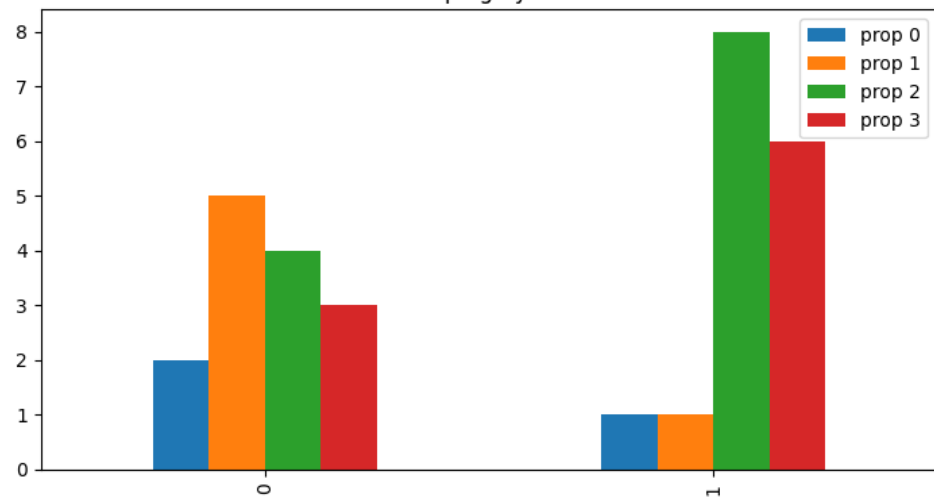
# using pandas dataframe
df = pd.DataFrame(m,columns=['prop 0','prop 1','prop 2','prop 3'])
df.plot(ax=ax[1,0],kind='bar')
ax[1,0].set_title('Grouping by rows')

# now other orientation (property X group)
ax[0,1].imshow(np.array(m).T)
df.T.plot(ax=ax[1,1],kind='bar')
ax[1,1].set_title('Grouping by columns')

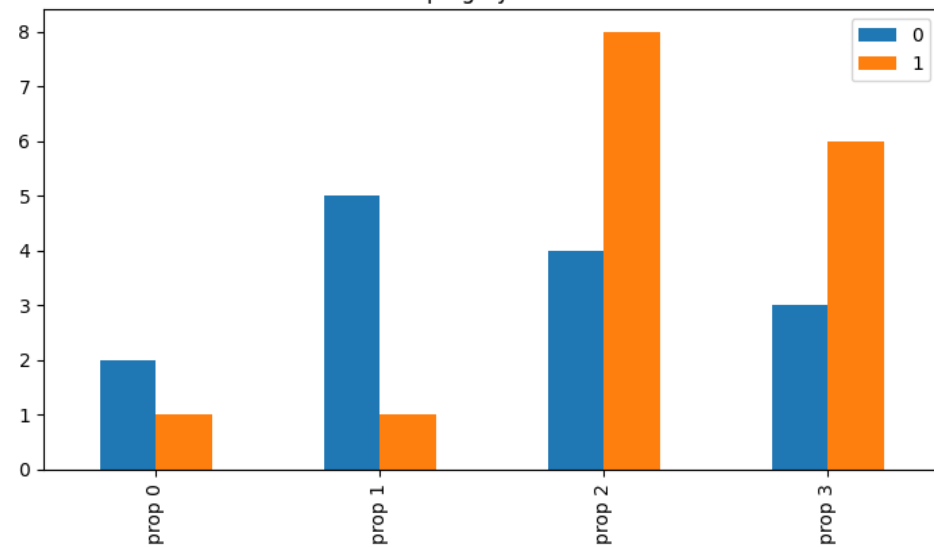
plt.show()
```



Grouping by rows



Grouping by columns

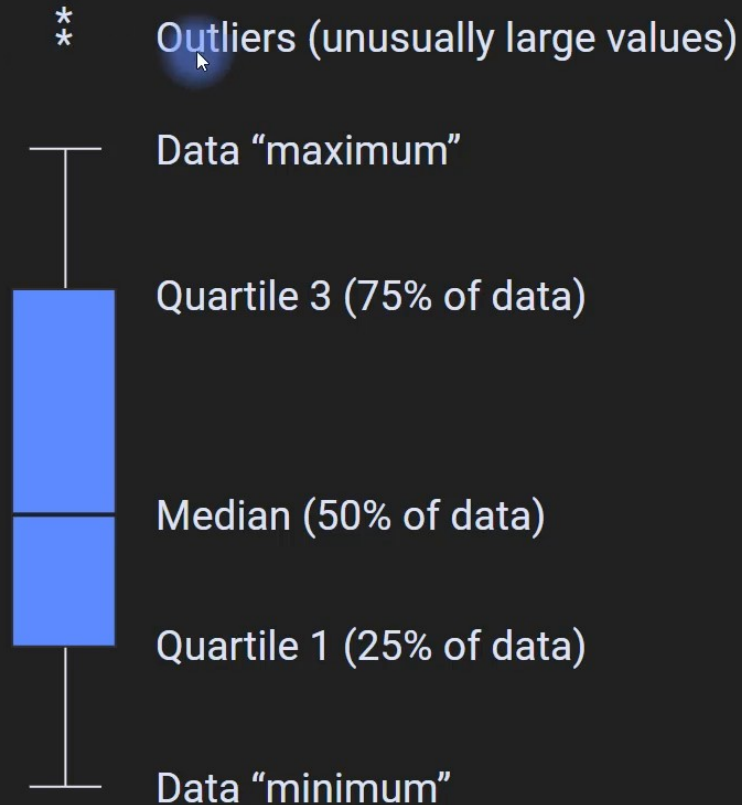


Box-and-whisker plots

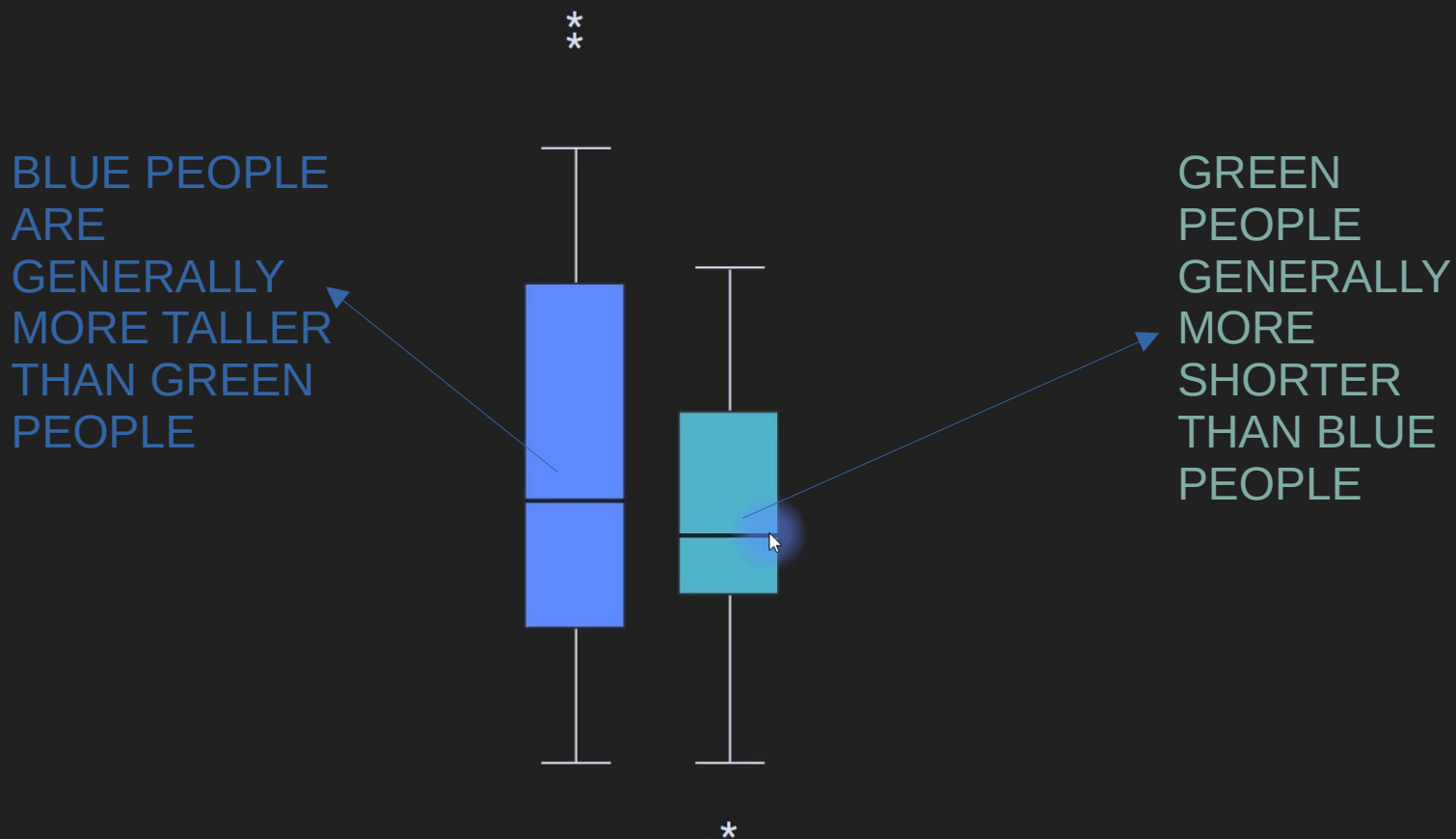
In this video, you will learn

- How to interpret box-and-whisker plots.
- The different markings on a box plot.

Box-and-whiskers plot (a.k.a. box plot)



Box-and-whiskers plot (a.k.a. box plot)



```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
```

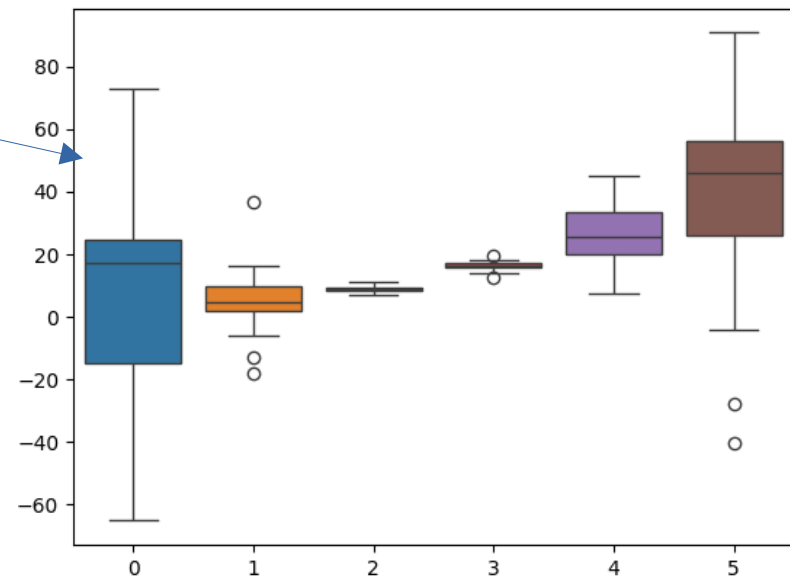
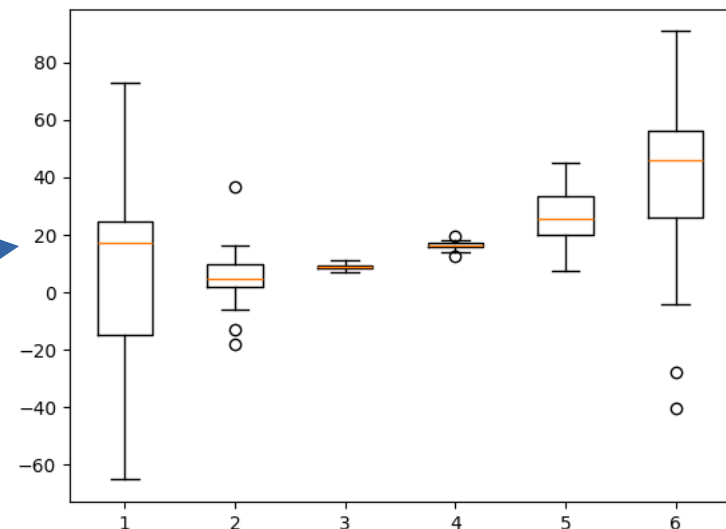
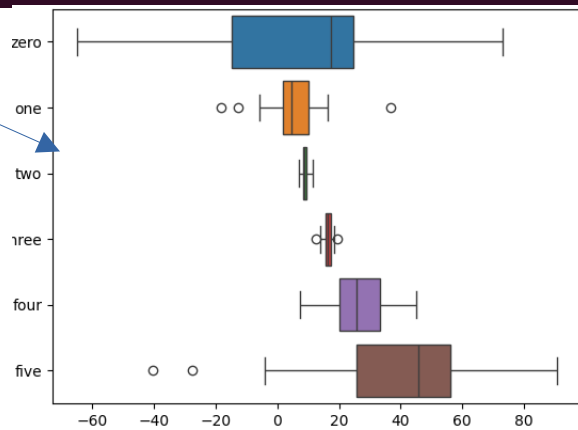
```
m = 30 # rows
n = 6 # columns
data = np.zeros((m,n))
for i in range(n):
    data[:,i] = 30*np.random.randn(m) * (2*i/(n-1)-1)**2 + (i+1)**2
```

```
# now for the boxplot
plt.boxplot(data)
plt.show()
```

```
# now with seaborn
sns.boxplot(data=data,orient='v')
plt.show()
```

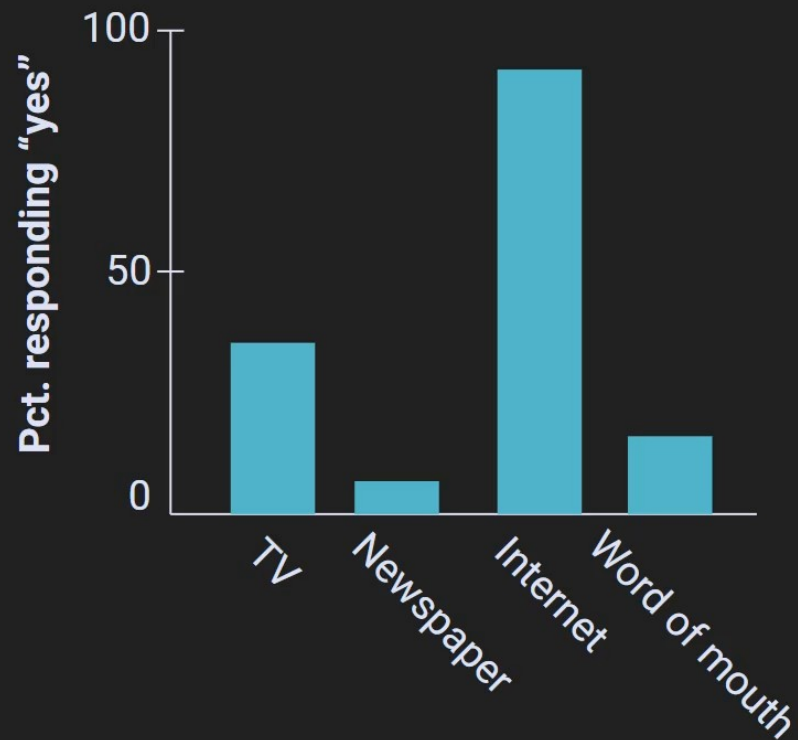
```
df = pd.DataFrame(data,columns=['zero','one','two','three','four','five'])
sns.boxplot(data=df,orient='h')
plt.show()
```

MEMORY PRE-ALLOCATION

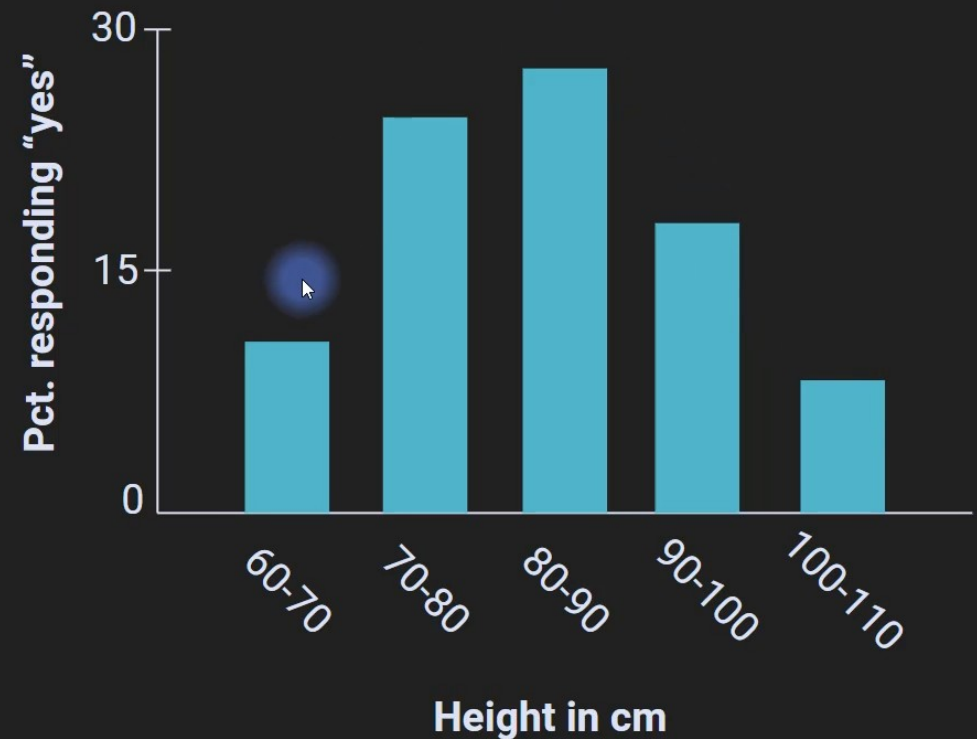


Histogram vs. bar plot

Bar plot

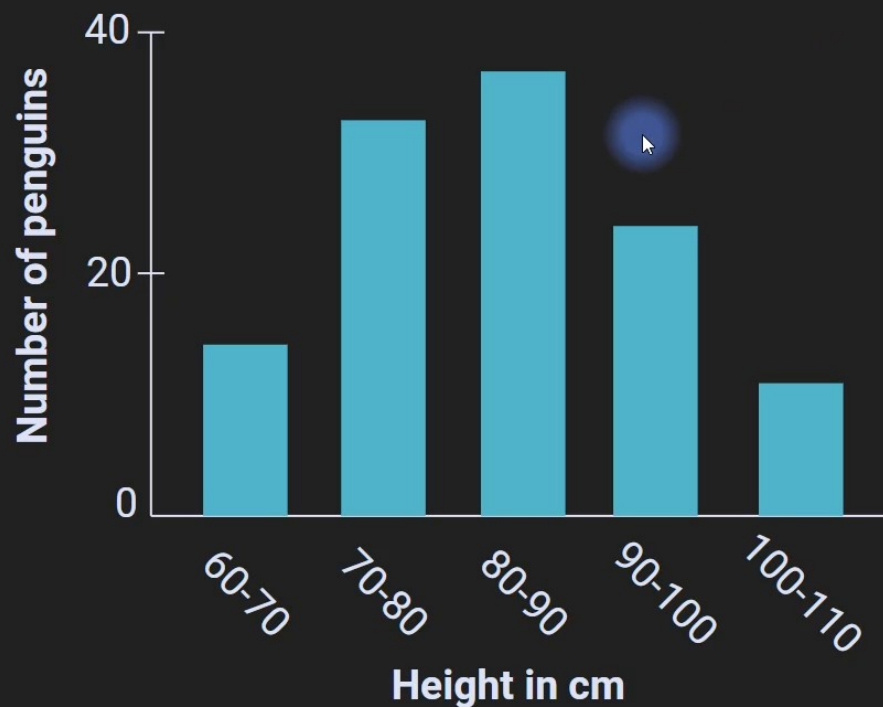


Histogram

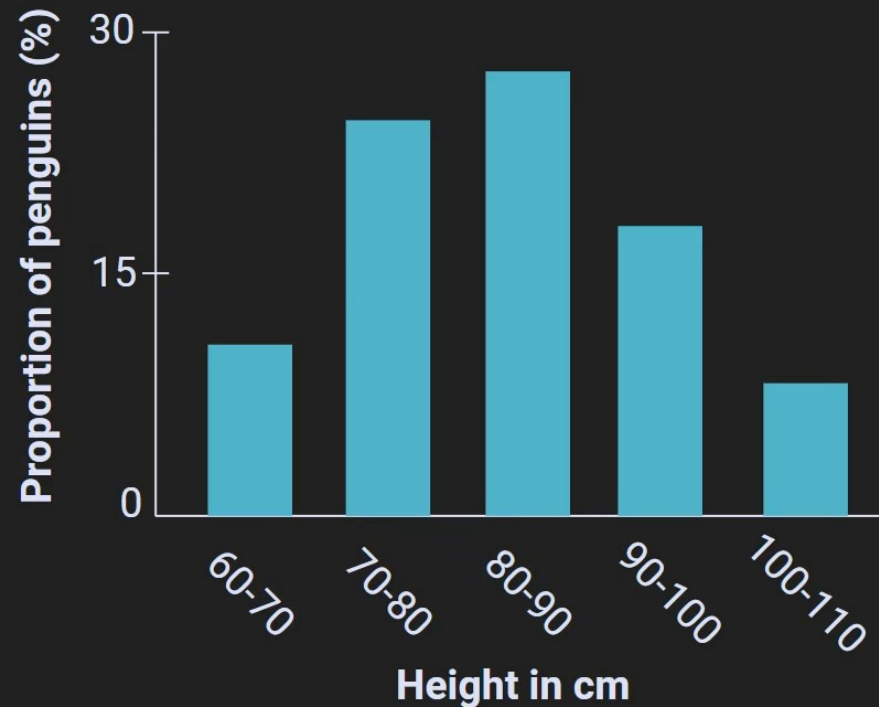


Histogram counts vs. proportion

Histogram of counts



Histogram of proportion



Histogram counts vs. proportion

Histogram of counts

Can be more meaningful to interpret (“raw” numbers).

Can be difficult to compare across datasets or studies.

Does not need to sum to 1 (or 100%).

Usually better for qualitative inspection.

Histogram of proportion

Can take extra effort to relate to the raw data.

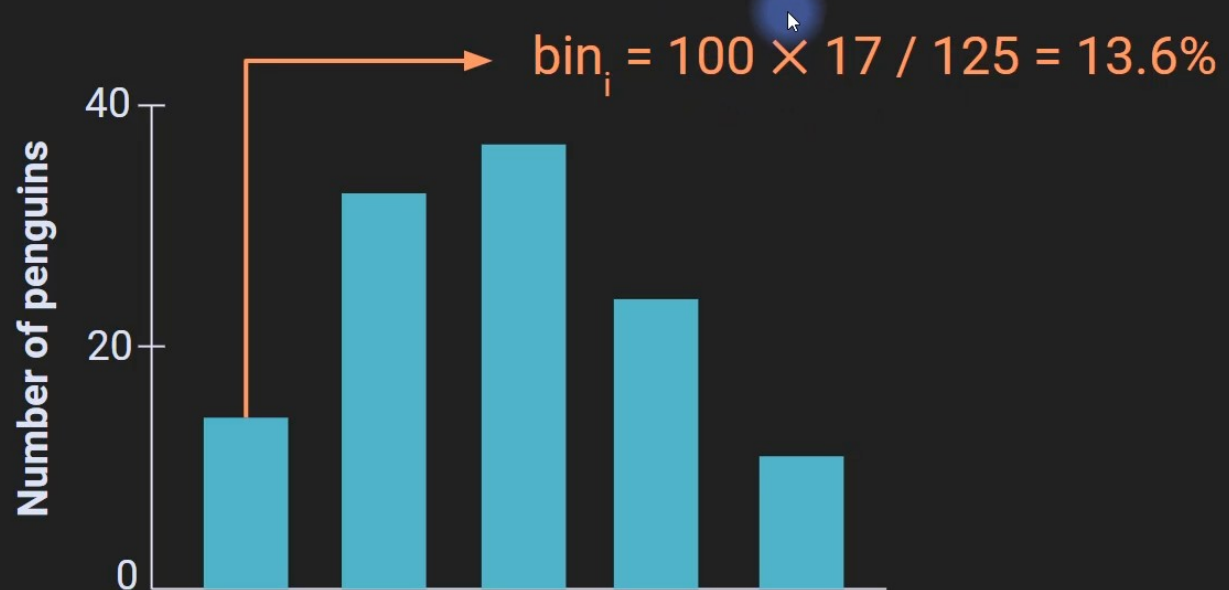
Easy to compare across datasets or studies.

Sums to 1 (or 100%).

Usually better for quantitative analysis.

Converting from counts to proportions

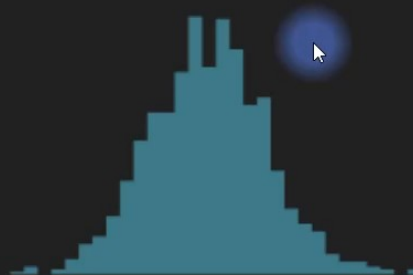
$$\text{bin}_i = 100 \times \text{bin}_i / \text{sum}(\text{bins})$$



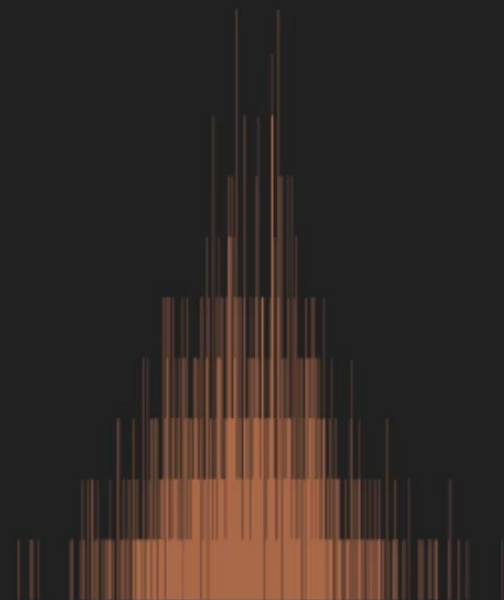
How many bins to have in a histogram?



4 bins



30 bins

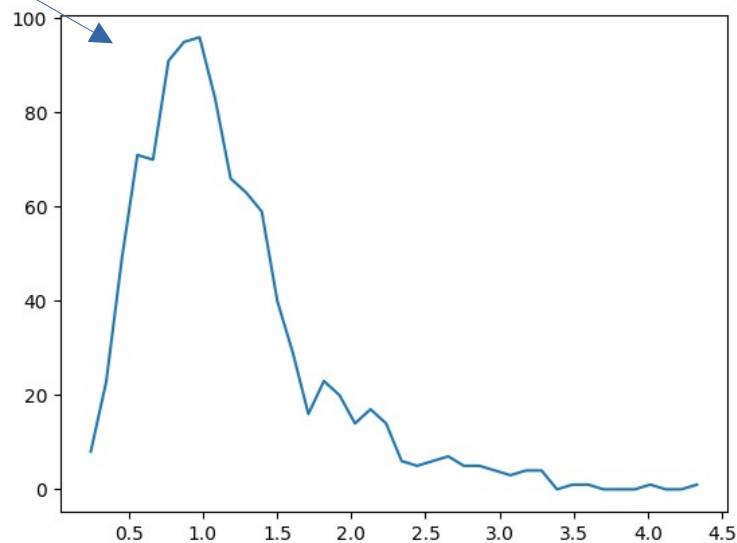
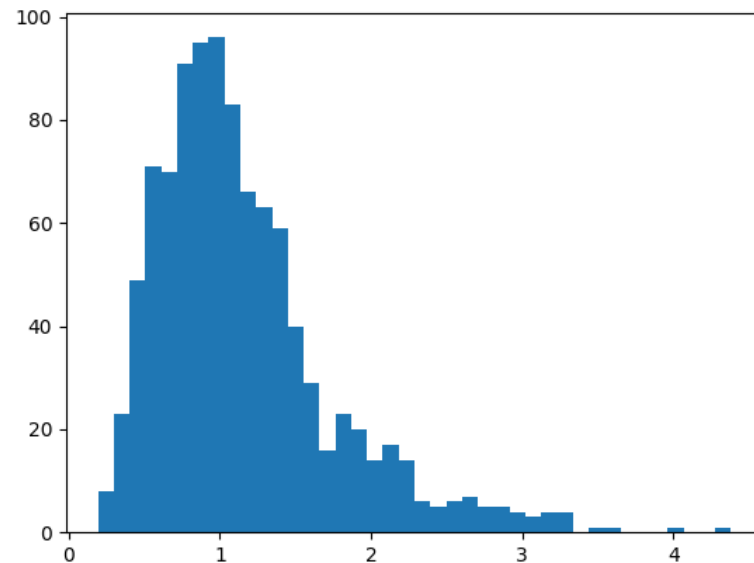


700 bins

```
import matplotlib.pyplot as plt
import numpy as np
n = 1000
data = np.exp( np.random.randn(n)/2 )

# number of histogram bins
k = 40
plt.hist(data,bins=k)
plt.show()

# another option
y,x = np.histogram(data,bins=k)
xx = (x[1:]+x[:-1])/2
plt.plot(xx,y)
plt.show()
```



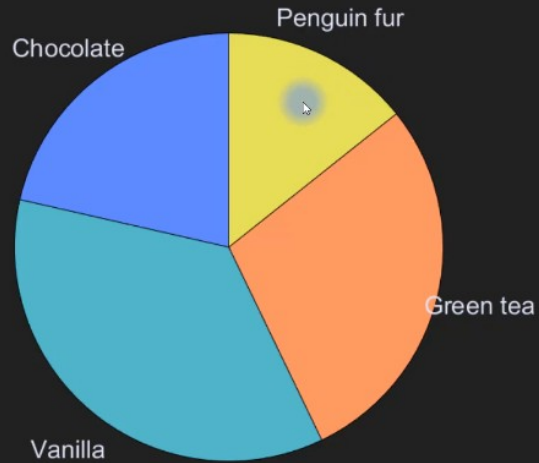
Pie charts

In this video, you will learn

- How to interpret pie charts.
- The types of data that can be used in pie charts.
- The condition on data for display in pie charts.

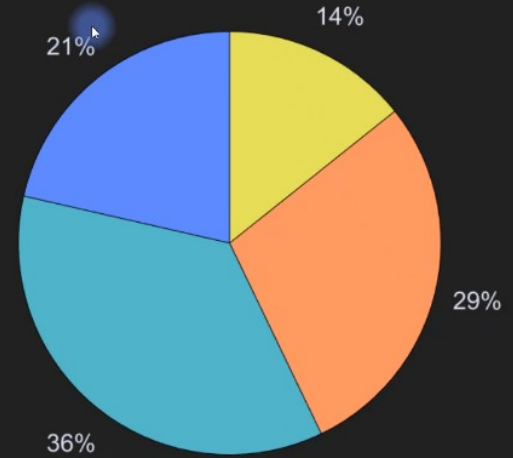
A piece of the pie

What is your favorite ice-cream flavor?



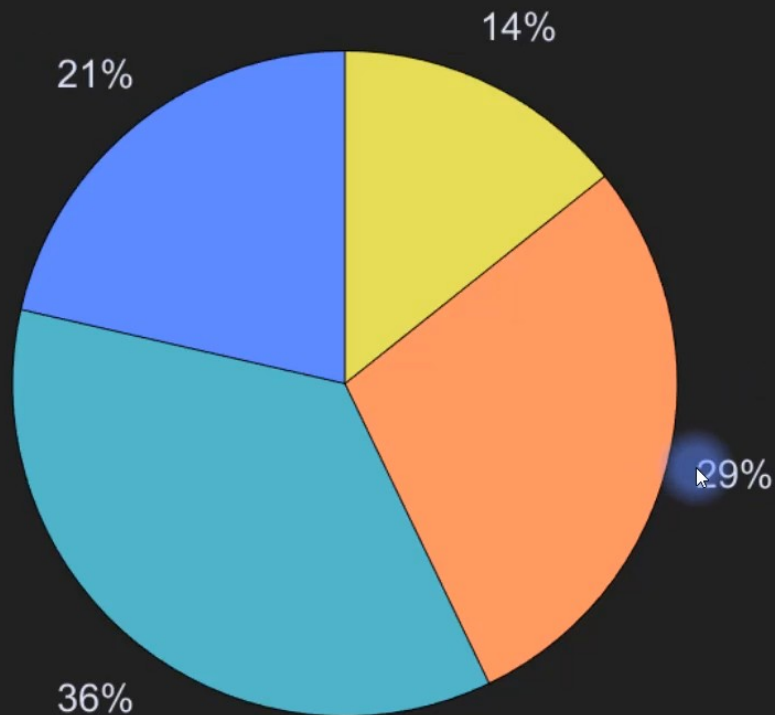
A piece of the pie

What is your favorite ice-cream flavor?



A piece of the pie

What is your favorite ice-cream flavor?



What type of data are these?

Nominal

What types of data can be shown in pie charts?

Nominal, ordinal, discrete

What is the condition on data for a pie chart?

Pie pieces must sum to 1 (or 100%).

```

import matplotlib.pyplot as plt
import numpy as np

nbins = 5
totalN = 100

rawdata = np.ceil(np.logspace(np.log10(1/2),
                             np.log10(nbins-.01),
                             totalN))

uniquenums = np.unique(rawdata)
data4pie = np.zeros(len(uniquenums))

for i in range(len(uniquenums)):
    data4pie[i] = sum(rawdata==uniquenums[i])

# show the pie chart
plt.pie(data4pie, labels=100*data4pie/sum(data4pie))
plt.show()

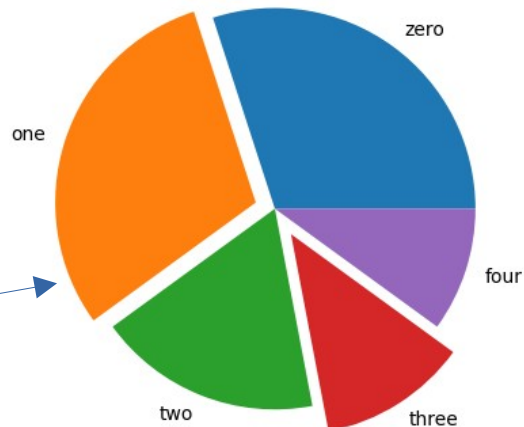
# another option
plt.pie(data4pie,
        labels=['zero', 'one', 'two', 'three', 'four'],
        explode=[0,.1,0,.15,0])
plt.show()

## for continuous data
# generate log-normal distribution
data = np.exp( np.random.randn(1000)/10 )

# generate bins using histogram
histout = np.histogram(data,bins=6)

# and show that as a pie chart
plt.pie(histout[0])
plt.show()

```



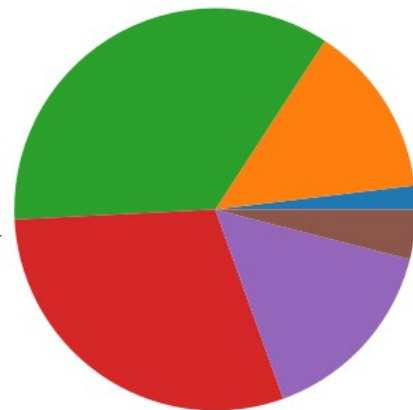
30.0

30.0

10.0

12.0

18.0

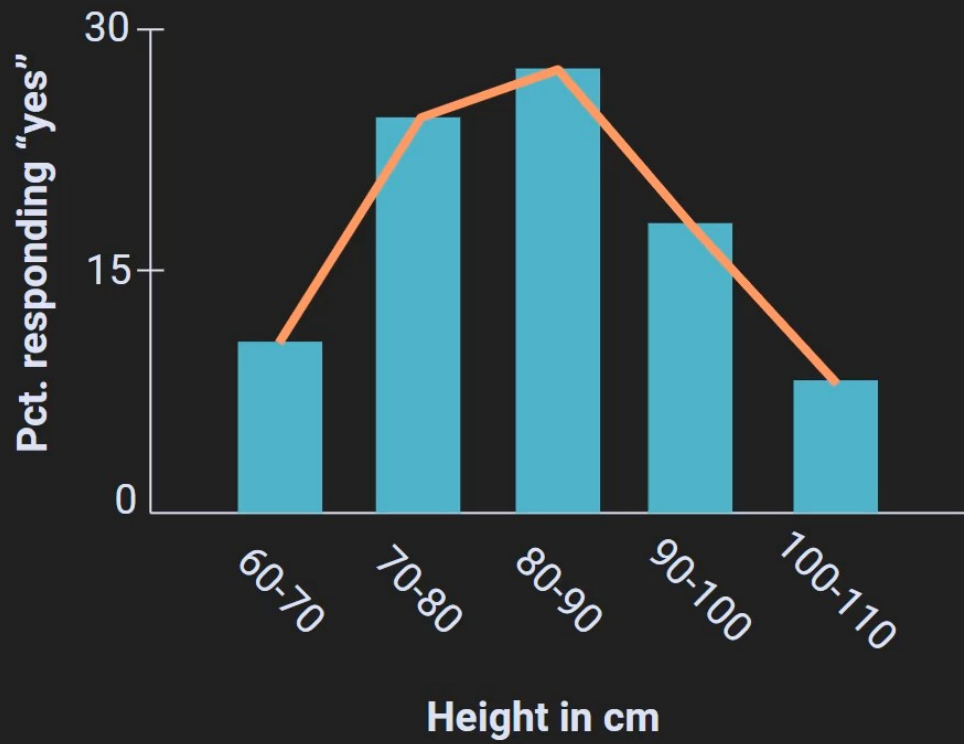
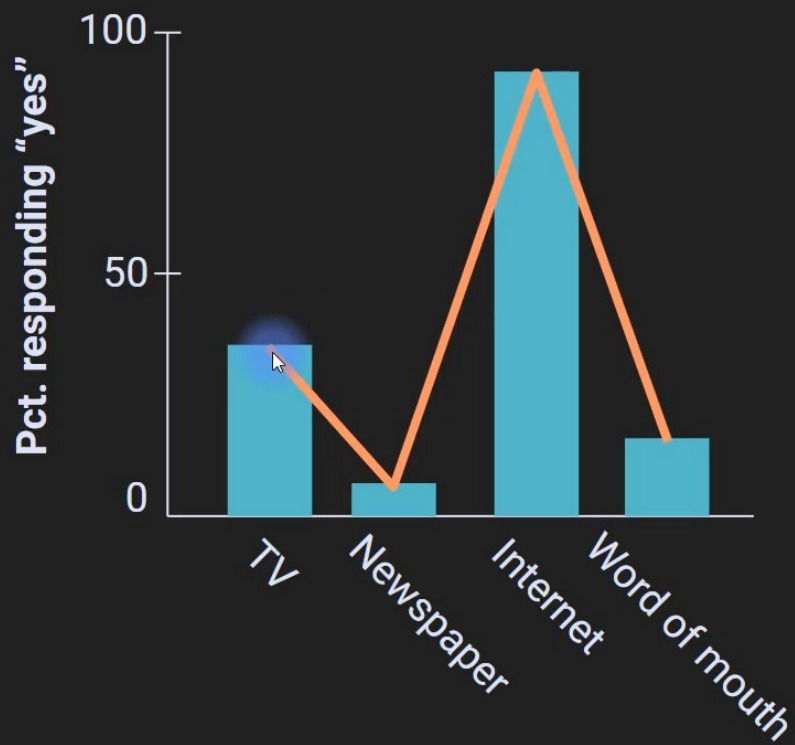


Using lines vs. bars

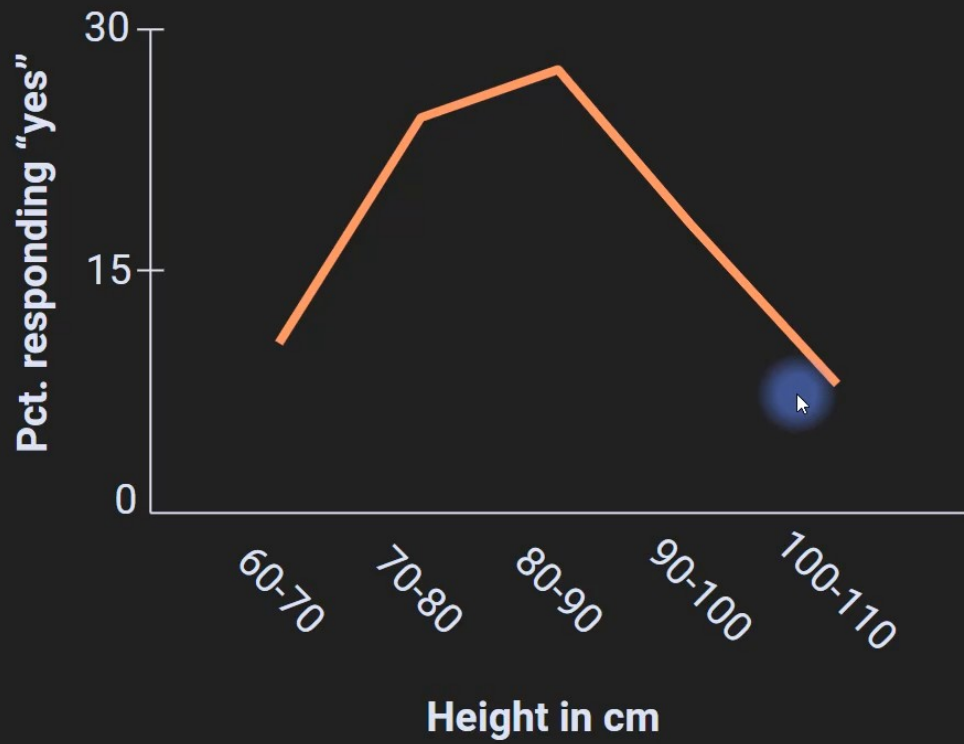
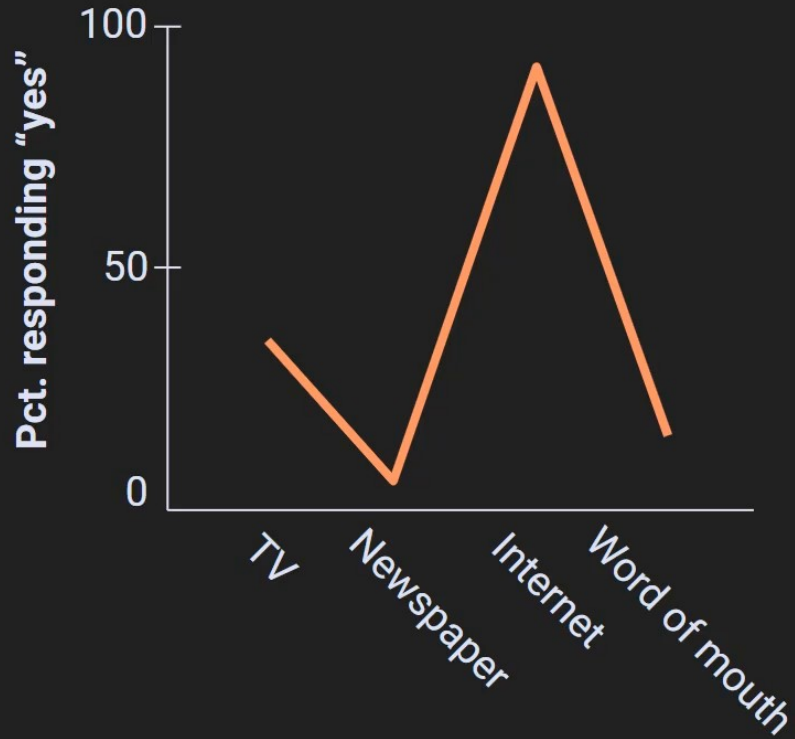
In this video, you will learn

- The implicit assumption behind line plots.
- Why it is almost always technically incorrect to use lines instead of bars.
- Why it is often practical to use lines instead of bars.

Lines vs. bars



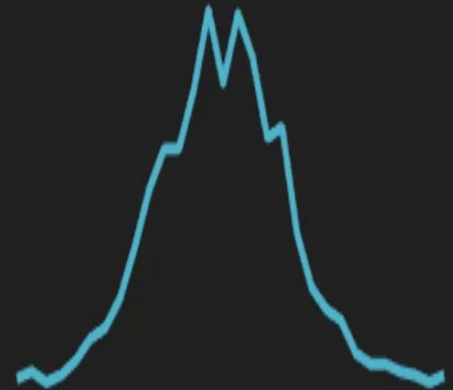
Lines vs. bars



Lines vs. bars

Two situations where lines can be used instead of bars:

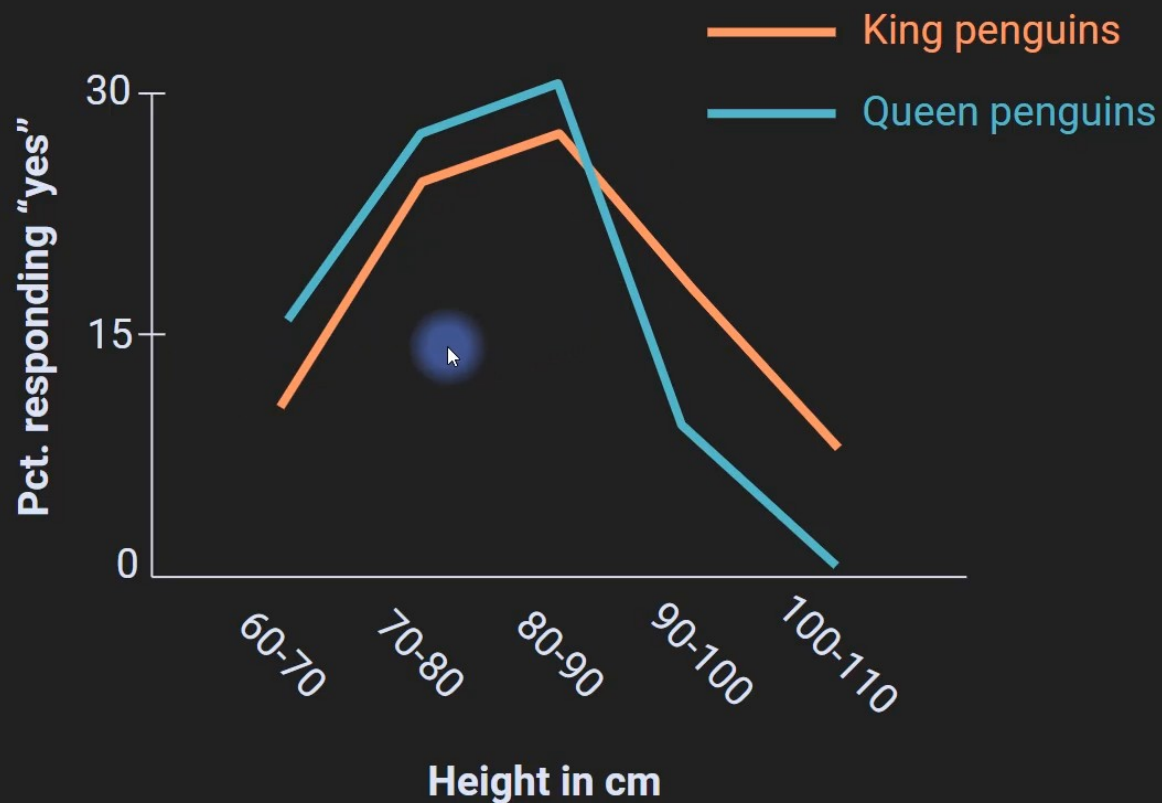
1) Many bars (fine resolution)



Lines vs. bars

Two situations where lines can be used instead of bars:

- 1) Many bars (fine resolution)
- 2) In-graph comparisons



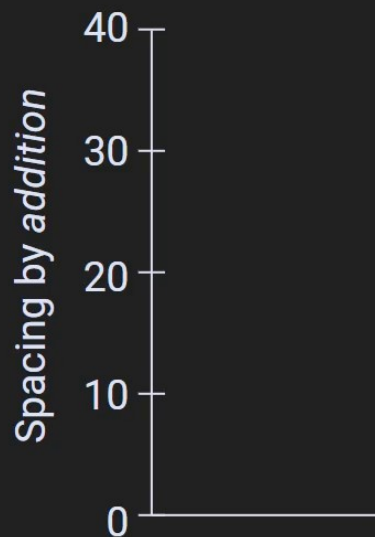
Linear vs. logarithmic axis scaling

In this video, you will learn

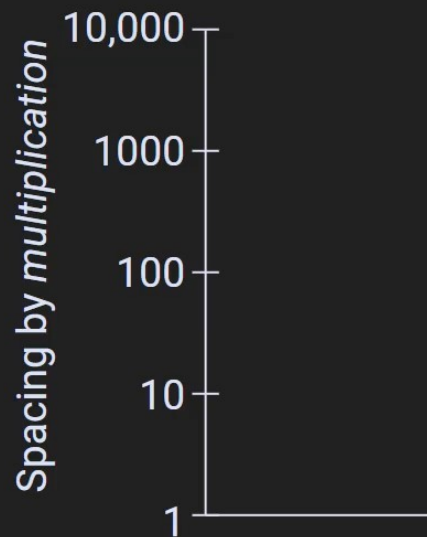
- How to create a logarithmic plot.
- How to interpret linear vs. log axes.
- When it's better to use log-scaled plots.

Linear vs. logarithmic axes

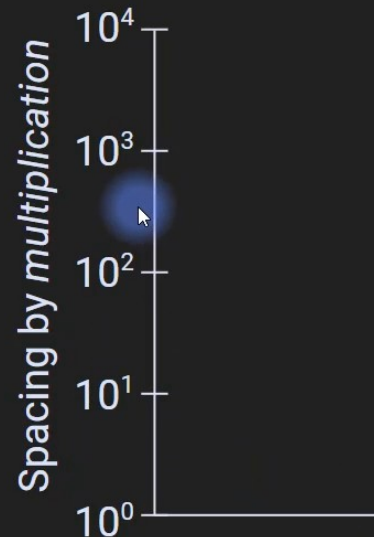
Linear y-axis scaling



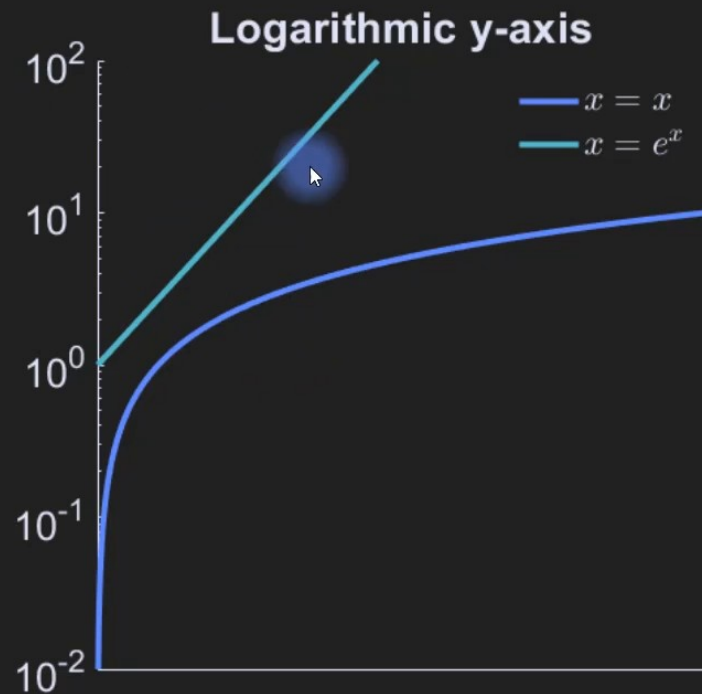
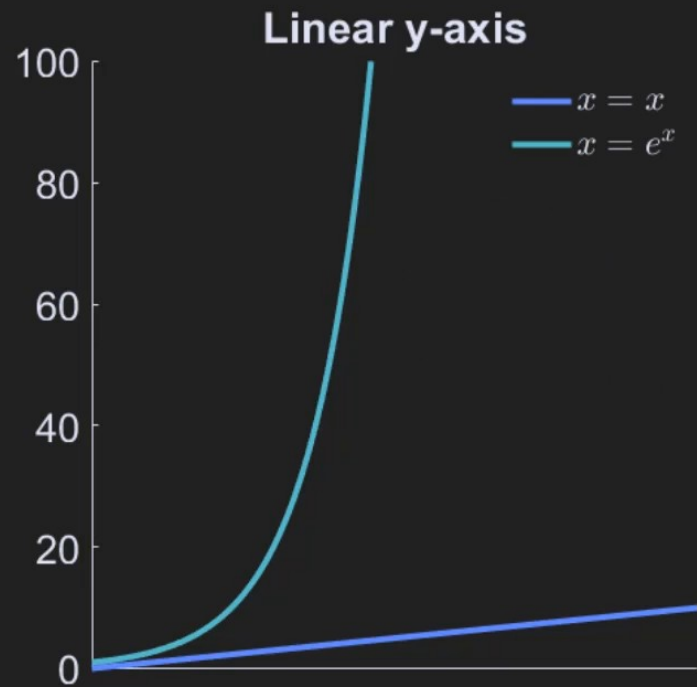
Logarithmic y-axis scaling



Log scaling



Lin-v-log



When to use which?

Linear scaling

Often easier to interpret.

Easily scaled to big, small, or negative numbers.

Can obscure trends or comparisons across variables.

Use unless you have a compelling reason to use log-plots.

Logarithmic scaling

Might need explanation for general audiences.

Might not work with negative numbers.

Often appropriate physics/biology, finance, growth, or really big numbers.

Use only when necessary.

```

import matplotlib.pyplot as plt
import numpy as np

# number of data points
n = 1000

# generate log-normal distribution
data1 = np.exp( np.random.randn(n)/2 )
data2 = np.exp( np.random.randn(n)/10 )
data3 = np.exp( np.random.randn(n)/2 + 1 )

k = 20

plt.hist(data1,bins=k)
plt.hist(data2,bins=k)
plt.hist(data3,bins=k)

plt.show()

# histogram discretization for the datasets
y1,x1 = np.histogram(data1,bins=k)
xx1 = (x1[0:-1] + x1[1:]) / 2
y1 = y1 / sum(y1) # convert to probability

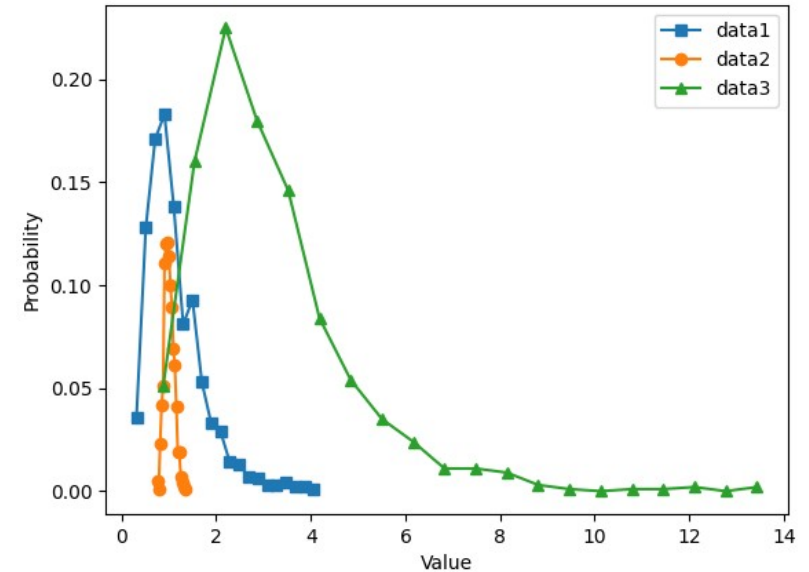
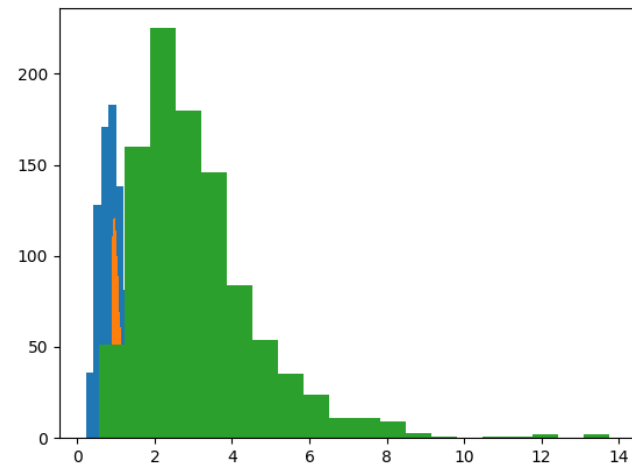
y2,x2 = np.histogram(data2,bins=k)
xx2 = (x2[0:-1] + x2[1:]) / 2
y2 = y2 / sum(y2) # convert to probability

y3,x3 = np.histogram(data3,bins=k)
xx3 = (x3[0:-1] + x3[1:]) / 2
y3 = y3 / sum(y3) # convert to probability

plt.plot(xx1,y1,'s-',label='data1')
plt.plot(xx2,y2,'o-',label='data2')
plt.plot(xx3,y3,'^-',label='data3')

plt.legend()
plt.xlabel('Value')
plt.ylabel('Probability')
plt.show()

```



<https://seaborn.pydata.org/examples/index.html>

<https://www.geeksforgeeks.org/python/python-seaborn-tutorial/>

https://seaborn.pydata.org/tutorial/function_overview.html

<https://www.geeksforgeeks.org/python/matplotlib-tutorial/>

<https://matplotlib.org/stable/gallery/index.html>

<https://github.com/matplotlib/matplotlib>